

Contents

Dynamic Programming — 1D — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	13
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	13
SECTION 18 — Dry Runs	15
SECTION 19 — Advanced Concepts	15
SECTION 20 — Senior Engineer Insights	16
SECTION 21 — Cheat Sheet	16
SECTION 22 — Revision Notes (5-Minute Review)	16
SECTION 23 — Practice Roadmap	16
SECTION 24 — Memory Tricks	17
SECTION 25 — Final Summary	17

Dynamic Programming — 1D — Complete Interview Handbook

Pattern #22 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Dynamic Programming section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

1D Dynamic Programming solves optimization/counting problems where the state can be fully described by a **single index** — $dp[i]$ represents “the answer considering only the first i elements” (or “ending at index i ,” or “using value i ”). The answer to the full problem is built from a **recurrence relation** expressing $dp[i]$ in terms of $dp[j]$ for $j < i$, exploiting **overlapping subproblems** (the same smaller subproblem is needed by multiple larger ones) and **optimal substructure** (the optimal solution to the whole problem is composed of optimal solutions to subproblems).

1.2 Why Was This Pattern Invented?

Pure recursion (as in Backtracking, Pattern #12) re-solves identical subproblems exponentially many times when subproblems overlap (e.g., computing $Fibonacci(5)$ recursively recomputes $Fibonacci(3)$ multiple times). DP was invented to **cache each unique subproblem’s answer once** — either top-down (memoization: recurse normally, but check a cache first) or bottom-up (tabulation: iteratively build up from the smallest subproblems) — converting exponential re-computation into polynomial (often linear) time.

1.3 Real Intuition Behind The Pattern

Imagine **climbing a staircase where you can take 1 or 2 steps at a time**, and you want to know how many distinct ways exist to reach step n . Computing this recursively re-asks “how many ways to reach step 5” many times as a subproblem of reaching step 6, step 7, etc. DP simply **writes down the answer for step 5 once**, and every future question about it becomes an instant lookup instead of a re-computation.

1.4 Mental Model

“What’s the smallest version of this problem I can solve directly (base case), and how does the answer for a slightly bigger version relate to smaller ones I’ve already solved?” Identify $dp[i]$ ’s meaning precisely, find the recurrence connecting it to $dp[i-1]$, $dp[i-2]$, etc., and fill the table in an order that guarantees dependencies are ready before they’re needed.

1.5 Visual Explanation

```
Climbing Stairs (n=5): dp[i] = number of ways to reach step i
dp[0] = 1 (base: one way to "reach" the ground - do nothing)
dp[1] = 1 (one way: single 1-step)
dp[i] = dp[i-1] + dp[i-2]      (arrive via a 1-step from i-1, OR a 2-step from i-2)
```

```
dp[2] = dp[1] + dp[0] = 1 + 1 = 2
dp[3] = dp[2] + dp[1] = 2 + 1 = 3
dp[4] = dp[3] + dp[2] = 3 + 2 = 5
dp[5] = dp[4] + dp[3] = 5 + 3 = 8
```

Table: [1, 1, 2, 3, 5, 8] → answer for $n=5$ is 8

1.6 Simple Analogy

1D DP is like **filling in a table of answers to increasingly bigger versions of the same question, left to right, where each new answer only needs to glance back at a few recent answers already written down** — never re-deriving anything from scratch.

1.7 When Should I Immediately Think About Using This Pattern?

- “Number of ways to...” (counting problems with a linear state).
- “Maximum/minimum ... ending at index i ” (optimization along a single sequence).
- “House Robber,” “Climbing Stairs,” “Maximum Subarray,” “Longest Increasing Subsequence.”
- A recursive brute-force solution has **overlapping subproblems** describable by a single index, and you’re being asked for the optimal value/count, not all solutions.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“number of ways to reach/make”	Counting DP
“maximum/minimum sum/value ending at”	Optimization DP
“can you rob houses without adjacent”	Constraint-based 1D DP
“longest increasing subsequence”	1D DP (or DP + Binary Search hybrid)
“maximum subarray”	1D DP (Kadane’s algorithm)

2.2 Hidden Hints

A brute-force recursive solution whose recursion tree has **repeated identical calls** (e.g., `solve(i)` called multiple times with the same i from different parent calls) is the definitive signal for DP — draw the recursion tree mentally (or on paper) and look for this repetition.

2.3 Interview Clues

Interviewer asks “can you avoid recomputing the same subproblem multiple times?” after you present an exponential recursive solution — the most direct possible hint toward memoization/tabulation.

2.4 Common Trick Words

“Optimal,” “minimum cost,” “maximum profit,” “count the number of distinct ways” — these all imply a single best/total value is needed, not an enumeration of all possibilities (which would be Backtracking instead).

2.5 What Interviewers Expect

Correct identification of the state (`dp[i]`’s precise meaning), correct recurrence relation with justified base cases, and the ability to convert between top-down memoization and bottom-up tabulation, plus space optimization (rolling variables) when only a constant number of previous states are needed.

2.6 When NOT To Use This Pattern

- Subproblems **don’t overlap** — if a recursive brute force has no repeated identical subproblem calls, memoization provides no benefit (it’s Divide and Conquer instead, like Merge Sort).
- The problem needs **all** solutions/combinations, not just an optimal value/count — that’s Backtracking (Pattern #12).
- The state genuinely requires **more than one dimension** to describe (e.g., “considering the first i items AND remaining capacity j ”) — that’s 2D DP or Knapsack DP (Patterns #23/#24).

SECTION 3 — Decision Framework

Does a brute-force recursive solution have OVERLAPPING SUBPROBLEMS (same call, same arguments, multiple times)?

No → Divide and Conquer is sufficient (no memoization needed); or reconsider approach entirely

Yes

Can the state be described by a SINGLE INDEX (e.g., "first i elements")?

Yes → USE 1D DYNAMIC PROGRAMMING

No - needs an additional dimension (capacity, second index, etc.)?

USE 2D DP / KNAPSACK DP (Patterns #23/#24) instead

Do you need ALL optimal solutions/paths, not just the optimal VALUE?

Yes → Layer BACKTRACKING on top of the DP table (reconstruct paths from recorded choices)

No → Standard DP suffices, returning just the final optimal value

Why: DP's entire value proposition is caching solutions to overlapping subproblems — without overlap, there's nothing to cache, and plain recursion (or divide and conquer) is already optimal. Once overlap is confirmed, the number of independent "dimensions" needed to uniquely describe a subproblem's state determines whether 1D DP suffices or a higher-dimensional variant is required.

SECTION 4 — Why This Pattern Works

Mathematical/Logical (Optimal Substructure): For 1D DP to apply, the problem must exhibit optimal substructure — the optimal solution to the problem of size n can be constructed from optimal solutions to strictly smaller subproblems ($dp[i]$ for $i < n$). This is what justifies the recurrence relation: if $dp[i-1]$ and $dp[i-2]$ are truly optimal for their respective subproblems, then combining them correctly (per the problem's specific combination rule) yields the truly optimal $dp[i]$.

Mathematical/Logical (Overlapping Subproblems + Memoization = Polynomial Time): Without memoization, a naive recursive solution following the same recurrence can require exponential time because $dp[i]$ may be recomputed once for every path in the recursion tree that reaches it (e.g., naive Fibonacci makes $O(2^n)$ calls). Memoization ensures each unique $dp[i]$ is computed exactly once, and every subsequent request for it is an $O(1)$ lookup — reducing total work to $O(n \times \text{cost-per-state})$.

Correctness Proof (by strong induction on i): *Base case:* $dp[0]$ (and other small base indices) are correct by direct problem definition (no recurrence needed). *Inductive hypothesis:* assume $dp[j]$ is correct for all $j < i$. *Inductive step:* the recurrence relation computes $dp[i]$ as a function of $dp[j]$ values for $j < i$ — since these are correct by the inductive hypothesis, and the recurrence correctly captures every way $dp[i]$ could arise from a valid smaller subproblem (this is where problem-specific reasoning is required — the recurrence must be proven exhaustive and non-overcounting), $dp[i]$ is therefore correct. *Termination:* by induction, $dp[n]$ (the final answer) is correct. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. **Define the state:** precisely state what $dp[i]$ represents (e.g., “the maximum sum of a sub-array ending at index i ”).
2. **Define the recurrence:** express $dp[i]$ in terms of $dp[j]$ for $j < i$ (often just $dp[i-1]$ and/or $dp[i-2]$).
3. **Define the base case(s):** the smallest index(es) where the answer is directly known without recursion.
4. **Choose direction:** top-down (memoized recursion) or bottom-up (iterative table-filling, usually simpler and avoiding recursion-depth limits).
5. **Optimize space:** if $dp[i]$ only depends on a constant number of previous states, replace the full array with a few rolling variables.

5.2 General Template — Bottom-Up Tabulation

```
function solve1DDP(n):
    dp = array of size n+1
    dp[0] = baseCase0
    dp[1] = baseCase1                # if needed

    for i in range(2, n+1):
        dp[i] = combine(dp[i-1], dp[i-2], ...) # problem-specific recurrence

    return dp[n]
```

5.3 General Template — Top-Down Memoization

```
memo = {}
function solve(i):
    if i in memo: return memo[i]
    if i is a base case: return baseCaseValue

    result = combine(solve(i-1), solve(i-2), ...)
    memo[i] = result
    return result
```

5.4 Space-Optimized Template (Rolling Variables)

```
function solve1DDPOptimized(n):
    prev2 = baseCase0
    prev1 = baseCase1
    for i in range(2, n+1):
        current = combine(prev1, prev2)
        prev2 = prev1
        prev1 = current
    return prev1
```

5.5 Interview Thinking Process

1. “Let me define what $dp[i]$ precisely means — this is the most important step.”
2. “I’ll write the brute-force recursive relation first, then identify which previous states it depends on.”
3. “I’ll verify the base cases are correct and sufficient to bootstrap the recurrence.”

4. “I’ll implement bottom-up (iterative) for clarity and to avoid recursion-depth issues, unless top-down is more natural for this specific recurrence.”
5. “I’ll note that space can be optimized to $O(1)$ if $dp[i]$ only depends on a constant number of previous entries.”

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$ to $O(n \times k)$ depending on recurrence complexity (k = number of prior states referenced)	$O(n)$ for the full table, $O(k)$ if space-optimized	Each of n states computed once, each taking $O(k)$ to combine prior states
Average Case	Same as worst — deterministic per-state cost	$O(n)$ or $O(1)$ optimized	No data-dependent variation typically
Best Case	$O(n)$ (must compute every state at least once)	$O(1)$ if only a constant window of history is needed	Even the “easiest” instance still requires building up to n
Amortized	$O(n)$ total — each state computed exactly once due to memoization	$O(n)$ or $O(1)$	This IS the entire benefit versus $O(2^n)$ naive recursion

SECTION 7 — Edge Cases

Edge Case	Example	Handling
$n = 0$	Empty sequence	Base case must handle this directly, often returning 0 or 1 depending on problem semantics
$n = 1$	Single element	Must not index $dp[i-2]$ out of bounds — guard base cases explicitly
Negative numbers (Maximum Subarray)	$[-2, 1, -3, 4]$	Kadane’s algorithm-style DP: $dp[i] = \max(arr[i], dp[i-1] + arr[i])$ correctly handles negative “reset” points
All negative numbers	$[-3, -1, -4]$	Maximum Subarray must still return the least-negative single element, not 0 (empty subarray isn’t always valid)
Recurrence referencing $dp[i-2]$ when $i=1$	Off-by-one at the boundary	Explicitly define $dp[-1]/dp[0]$ conceptually or guard with conditionals

Edge Case	Example	Handling
Very large n (10^6+)	Stack overflow risk with naive top-down recursion	Prefer bottom-up iteration, or increase recursion limits / convert to iterative explicitly
Ties in optimal value	Multiple subsequences with the same max value	Clarify whether the problem needs the count of optimal solutions or just the value itself

Common mistakes: off-by-one errors when indexing $dp[i-1]/dp[i-2]$ near the array's start; forgetting that "maximum subarray" problems require handling the case where all numbers are negative (the answer isn't simply 0, an empty subarray often isn't valid); using top-down recursion without memoization by mistake (accidentally re-deriving the exponential blowup DP was meant to eliminate).

SECTION 8 — Pros & Cons

Advantages: Converts exponential brute-force recursion into polynomial (often linear) time; conceptually reusable across a huge family of "optimal value along a sequence" problems; often reducible to $O(1)$ space with rolling variables. **Disadvantages:** Requires correctly identifying the state and recurrence, which can be non-obvious for some problems; top-down memoization risks stack overflow for very large n ; doesn't directly enumerate all optimal solutions (needs backtracking layered on top if that's required). **Trade-offs:** Top-down memoization (more intuitive derivation from the recursive brute force, risk of stack overflow) vs. bottom-up tabulation (requires figuring out correct iteration order upfront, but avoids recursion limits and is often easier to space-optimize). **Limitations:** Only applicable when overlapping subproblems and optimal substructure both hold; doesn't apply to problems requiring exhaustive enumeration. **Inefficient when:** subproblems don't actually overlap (no benefit over plain recursion/divide-and-conquer), or when the "1D" state assumption is actually insufficient (needing a 2D or higher-dimensional formulation instead).

SECTION 9 — Real World Applications

Domain	Application
Finance	Optimal stock trading strategy computation (best time to buy/sell) using 1D DP over price sequences
Bioinformatics	Sequence alignment scoring (simplified 1D variants), mutation-distance estimation
Compilers	Optimal instruction scheduling along a linear instruction sequence (simplified DP formulations)
Resource Allocation	Optimal budget allocation across sequential time periods with carry-over constraints
Robotics	Optimal path cost computation along constrained 1D movement sequences
Speech Recognition	Simplified sequence-alignment-style scoring in early-stage signal processing pipelines

Domain	Application
Text Processing	Word-wrap/justification algorithms computing minimum “badness” line-break points along a sequence of words
Networking	Optimal buffer allocation along a sequential packet-processing pipeline
Game Development	Optimal scoring/path computation in linear level-progression mechanics
Operations Research	Sequential decision-making problems (inventory management over time) modeled as 1D DP

SECTION 10 — Interview Strategy

How seniors answer: They start by precisely defining $dp[i]$ ’s meaning in plain English before writing any code, derive the recurrence by reasoning about “how can I arrive at state i from a smaller state,” verify base cases explicitly, and proactively mention the $O(1)$ space optimization when applicable.

How juniors answer: They often jump straight to code without stating the state definition, leading to recurrences that are subtly wrong or inconsistent, or they implement a correct DP but fail to recognize the space-optimization opportunity, leaving an unnecessary $O(n)$ array when $O(1)$ would do.

Typical follow-ups: “Can you optimize the space to $O(1)$?” “Can you show the top-down (memoized) version too?” “How would this change if [some added constraint]?” “What’s the recursion tree look like without memoization, and why is it exponential?”

Optimization questions: “Can you avoid the extra array entirely?” (rolling variables); “Is there a closed-form/mathematical shortcut instead of DP?” (occasionally, e.g., Fibonacci has a closed form via Binet’s formula, though DP is usually still the expected interview answer).

SECTION 11 — Pattern Variations

Variation	Description	Example
Simple Counting DP	$dp[i]$ = number of ways to reach state i	Climbing Stairs, Fibonacci Number
Constraint-Based Optimization	$dp[i]$ = best value respecting a constraint (e.g., no adjacent selection)	House Robber
Kadane’s Algorithm (Running Max)	$dp[i]$ = best value ending exactly at i , with reset logic	Maximum Subarray, Maximum Product Subarray
Longest Increasing Subsequence Family	$dp[i]$ = length of best subsequence ending at i	Longest Increasing Subsequence
Decision DP (Include/Exclude)	$dp[i]$ = best value considering whether to include element i	House Robber, Delete and Earn
DP + Binary Search Hybrid	Maintaining a sorted “tails” array for $O(n \log n)$ LIS	Longest Increasing Subsequence (optimized)

Variation	Description	Example
DP with State Compression (still 1D over positions)	dp[i] tracks best value AND a secondary piece of info together	Best Time to Buy and Sell Stock with Cooldown/Fee

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Backtracking	Enumerates ALL solutions exhaustively, no caching of overlapping subproblems	Need every valid combination/permutation, not just the optimal value
2D DP / Knapsack DP	State needs TWO dimensions to describe (e.g., index + remaining capacity)	Problem inherently has two varying quantities
Greedy	Makes a single locally-optimal choice per step, no subproblem table maintained	A provable exchange argument shows local optimality implies global optimality (no need to consider alternatives)
Divide and Conquer	Splits into independent (non-overlapping) subproblems	Subproblems don't share/overlap — no benefit from memoization

Comparison Table

Aspect	1D DP	Backtracking	Greedy
Explores all possibilities	No (only tracks optimal per state)	Yes	No (single path)
Time complexity	Polynomial (often $O(n)$)	Exponential	Polynomial (often $O(n \log n)$ or $O(n)$)
Requires overlapping subproblems	Yes	N/A	N/A
Use case	Optimal value/count along a sequence	Enumerate all valid configurations	Single greedy choice provably optimal

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Simple linear recurrence, single base case chain	Climbing Stairs, Fibonacci Number, House Robber
Medium	Multiple states tracked simultaneously, decision-based recurrences	Maximum Subarray, House Robber II, Coin Change

Difficulty	Characteristics	Examples
Hard	DP combined with binary search or additional optimization	Longest Increasing Subsequence ($O(n \log n)$), Best Time to Buy and Sell Stock with Cooldown
Very Hard	Multi-state compressed DP, advanced recurrence design	Longest Valid Parentheses, Wiggle Subsequence variants at scale, Maximum Alternating Subsequence Sum

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Climbing Stairs	Easy	Amazon, Meta, Microsoft, Google	Direct Fibonacci-style 1D DP	Foundational recurrence mechanics
2	Fibonacci Number	Easy	Amazon, Meta	Classic overlapping subproblem introduction	Overlap recognition
3	House Robber	Medium	Amazon, Meta, Google, Microsoft	Include/exclude decision DP	Decision-based recurrence
4	House Robber II	Medium	Amazon, Meta	Circular constraint variant (two linear DP runs)	Constraint adaptation technique
5	Maximum Subarray	Medium	Amazon, Meta, Google, Microsoft	Kadane's algorithm (running max with reset)	Running-max DP mastery
6	Maximum Product Subarray	Medium	Amazon, Meta, Google	Dual running max/min DP (handles sign flips)	Dual-state running DP
7	Longest Increasing Subsequence	Medium	Amazon, Meta, Google, Microsoft	Classic 1D DP ($O(n^2)$) or DP+Binary Search ($O(n \log n)$)	Complexity optimization mastery
8	Coin Change	Medium	Amazon, Meta, Google, Microsoft	1D DP over amount, minimizing coin count	Minimization DP with unbounded choices

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
9	Coin Change II	Medium	Amazon, Google	1D DP counting combinations (unbounded knapsack framing)	Counting DP with unbounded choices
10	Decode Ways	Medium	Amazon, Meta, Google, Microsoft	1D DP with conditional branching recurrence	Conditional recurrence design
11	Word Break	Medium	Amazon, Meta, Google, Microsoft	1D DP with dictionary-lookup recurrence	Cross-pattern (DP + Hashing/Trie)
12	Delete and Earn	Medium	Amazon, Google	Transform into House Robber via frequency bucketing	Problem transformation technique
13	Longest Valid Parentheses	Hard	Amazon, Google, Meta	1D DP tracking valid-match lengths	Advanced string-based 1D DP
14	Best Time to Buy and Sell Stock	Easy	Amazon, Meta, Google, Microsoft	Simple running min/max DP	Foundational running-extremum DP
15	Best Time to Buy and Sell Stock with Cooldown	Medium	Amazon, Google	Multi-state 1D DP (holding/not-holding/cooldown)	Advanced multi-state DP
16	Best Time to Buy and Sell Stock with Transaction Fee	Medium	Amazon, Google	Multi-state 1D DP with fee adjustment	Multi-state DP variant
17	Perfect Squares	Medium	Amazon, Google	1D DP minimizing count of perfect squares summing to n	Minimization DP with combinatorial choices
18	Integer Break	Medium	Google, Amazon	1D DP maximizing product via optimal partitioning	Product-maximization DP
19	Jump Game	Medium	Amazon, Meta, Google	1D DP or Greedy (reachability)	Cross-pattern (DP vs Greedy comparison)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
20	Jump Game II	Medium	Amazon, Meta, Google	1D DP or Greedy (minimum jumps)	Cross-pattern (DP vs Greedy comparison)
21	Partition Equal Subset Sum (contrast, Knapsack)	Medium	Amazon, Meta, Google	Contrast: 0/1 Knapsack DP, not pure 1D	Pattern-boundary awareness
22	Wiggle Subsequence	Medium	Amazon, Google	Dual-state 1D DP (up/down tracking)	Dual-state DP mastery
23	Maximum Alternating Subsequence Sum	Medium	Google	Dual-state running DP	Advanced dual-state DP
24	Min Cost Climbing Stairs	Easy	Amazon, Google	Simple 1D DP with cost minimization	Foundational minimization DP
25	Domino and Tromino Tiling	Hard	Google	Advanced multi-term recurrence 1D DP	Advanced recurrence design
26	Count Ways to Reach Nth Stair (variant with additional moves)	Medium	Amazon	Generalized Climbing Stairs recurrence	Recurrence generalization
27	Non-negative Integers without Consecutive Ones (bit-DP framing)	Hard	Google	1D DP over bit positions	Cross-pattern (Bit reasoning + DP)
28	Longest Turbulent Subarray	Medium	Amazon, Google	Dual-state running DP (increasing/decreasing)	Dual-state DP application
29	Divisor Game (contrast, Game Theory DP)	Easy	Google	Simple 1D DP with game-theoretic recurrence	Game theory DP introduction
30	Stone Game (contrast, interval/game DP)	Medium	Google, Amazon	Contrast: interval DP, not pure 1D	Pattern-boundary awareness

SECTION 15 — Common Mistakes

1. Not precisely defining what `dp[i]` means before writing the recurrence, leading to inconsistent or incorrect logic. *Fix:* always state the state definition in plain English first.
2. Off-by-one indexing errors, especially when the recurrence references `dp[i-2]` near the array's start. *Fix:* explicitly handle/guard small indices as distinct base cases.
3. Using top-down memoization without actually adding the memo cache (accidentally reverting to plain exponential recursion). *Fix:* always verify the memo check is present and correctly keyed.
4. Missing the $O(1)$ space optimization opportunity when only a constant window of previous states is needed. *Fix:* after getting a correct $O(n)$ -space solution, always ask “do I really need the full array, or just the last few values?”
5. Confusing 1D DP with problems that actually need a second dimension (e.g., trying to force a Knapsack-style problem into a single-index recurrence, losing critical state information like remaining capacity). *Fix:* carefully verify the state truly needs only one dimension before committing to a 1D formulation.

Why people fail: the state definition step is deceptively easy to skip since the code often “looks” plausible even with a slightly wrong recurrence — candidates who don’t rigorously verify their recurrence against the base cases and a hand-traced small example often ship subtly incorrect DP solutions that pass some test cases but fail edge cases.

SECTION 16 — Optimization Techniques

- **Time:** For Longest Increasing Subsequence and similar problems, recognize when a DP + Binary Search hybrid can reduce $O(n^2)$ to $O(n \log n)$ by maintaining a sorted “best tails” array instead of a full pairwise comparison.
 - **Space:** Always check whether the recurrence only references a constant number of previous states (`dp[i-1]`, `dp[i-2]`) — if so, replace the full array with rolling variables for $O(1)$ space.
 - **Readability:** Name DP arrays/variables semantically (`waysToReach`, `maxEndingHere`) rather than generic `dp`; comment the state definition directly above the recurrence.
 - **Interview performance:** Always state the state definition and recurrence in words before coding — this is the single highest-signal habit for any DP problem, 1D or otherwise.
-

SECTION 17 — Multiple Language Templates

Java

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

JavaScript

```
function climbStairs(n) {
  if (n <= 2) return n;
  let prev2 = 1, prev1 = 2;
  for (let i = 3; i <= n; i++) {
    const current = prev1 + prev2;
    prev2 = prev1;
    prev1 = current;
  }
  return prev1;
}
```

PHP

```
function climbStairs(int $n): int {
  if ($n <= 2) return $n;
  $prev2 = 1; $prev1 = 2;
  for ($i = 3; $i <= $n; $i++) {
    $current = $prev1 + $prev2;
    $prev2 = $prev1;
    $prev1 = $current;
  }
  return $prev1;
}
```

Python

```
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for i in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

Go

```
func climbStairs(n int) int {
  if n <= 2 {
    return n
  }
  prev2, prev1 := 1, 2
  for i := 3; i <= n; i++ {
    current := prev1 + prev2
    prev2 = prev1
    prev1 = current
  }
  return prev1
}
```

C++

```

int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}

```

SECTION 18 — Dry Runs

Small Input

$n = 5$ (Climbing Stairs)

```

prev2=1(dp[1]), prev1=2(dp[2])
i=3: current=2+1=3 → prev2=2, prev1=3    (dp[3]=3)
i=4: current=3+2=5 → prev2=3, prev1=5    (dp[4]=5)
i=5: current=5+3=8 → prev2=5, prev1=8    (dp[5]=8)
Result: 8

```

Large Input (Conceptual)

For $n = 10^6$, the space-optimized iterative solution performs exactly 10^6 additions with $O(1)$ memory — versus a naive unmemoized recursive solution which would attempt $O(2^{10^6})$ calls, utterly infeasible; this stark contrast is exactly why DP's caching is essential.

Corner Case

$n = 0$: return 0 directly (no steps needed, trivially “1 way” or “0 ways” depending on exact problem framing — clarify with the interviewer). $n = 1$: return 1 directly (only one way: a single 1-step) — must guard this base case before the main loop executes to avoid referencing $dp[-1]$.

SECTION 19 — Advanced Concepts

- **DP + Binary Search hybrid (Longest Increasing Subsequence):** instead of the $O(n^2)$ $dp[i] = 1 + \max(dp[j])$ for all $j < i$ with $arr[j] < arr[i]$, maintain a separate array of “smallest possible tail value for an increasing subsequence of each length,” updated via binary search — this reduces LIS to $O(n \log n)$, a non-obvious but powerful and frequently-tested optimization.
- **Multi-state 1D DP (Buy/Sell Stock variants):** some 1D DP problems actually track **multiple parallel states per index** (e.g., “max profit if holding a stock,” “max profit if not holding, in cooldown,” “max profit if not holding, not in cooldown”) — this is still “1D” in the sense that the index is the only structural dimension, but the state at each index is a small tuple, not a single scalar.
- **Problem transformation (Delete and Earn → House Robber):** recognizing that a seemingly different problem reduces to a well-known 1D DP pattern after a transformation (e.g., bucketing values by frequency) is a key advanced skill — Delete and Earn becomes exactly House Robber once you aggregate points by value and note that taking value v forbids taking $v-1$ and $v+1$.

- **Space-time trade-off awareness:** while $O(1)$ space optimization is usually presented as strictly better, mention that keeping the full dp array can be necessary if the problem later requires path reconstruction (which value choices led to the optimal answer) — a common follow-up.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize 1D DP as the simplest instance of a much broader technique — **caching solutions to overlapping subproblems to convert exponential recomputation into polynomial time** — the same principle underlies memoized API responses, dynamic programming in compiler optimization (common subexpression elimination), and reinforcement learning value-iteration algorithms. They evaluate whether a candidate can precisely state the state definition and recurrence *before* coding (rather than deriving it through trial-and-error), and whether they recognize the $O(1)$ space optimization and the DP+Binary Search hybrid opportunities where applicable — these are the concrete signals of deep, transferable DP fluency versus memorized problem-specific solutions.

SECTION 21 — Cheat Sheet

PATTERN: 1D Dynamic Programming

RECOGNIZE: "number of ways," "max/min ... ending at/using," overlapping subproblems describable by ONE in

TEMPLATE (bottom-up):

```
dp[0], dp[1] = base cases
for i in range(2, n+1):
    dp[i] = combine(dp[i-1], dp[i-2], ...) # problem-specific recurrence
return dp[n]
```

COMPLEXITY: $O(n)$ time typically; $O(n)$ space, often reducible to $O(1)$ with rolling variables

KEY PROOF: optimal substructure (bigger optimal built from smaller optimal) + overlapping subproblems (ca

WATCH FOR: precise state definition first, off-by-one at small indices, $O(1)$ space optimization opportuni

DOESN'T APPLY WHEN: no overlapping subproblems (plain recursion fine), need ALL solutions (Backtracking),

SECTION 22 — Revision Notes (5-Minute Review)

- 1D DP: $dp[i]$ = answer using only the first i elements (or ending at i); build via a recurrence from smaller states.
 - Always define the state in plain English BEFORE deriving the recurrence.
 - Base cases must be correct and sufficient to bootstrap the recurrence without out-of-bounds references.
 - Top-down (memoized recursion) vs bottom-up (iterative table) — bottom-up avoids recursion-depth risk.
 - Space-optimize to $O(1)$ with rolling variables whenever only a constant window of history is needed.
 - Kadane's algorithm (Maximum Subarray) handles negative numbers via a "reset" comparison: $\max(arr[i], dp[i-1]+arr[i])$.
 - LIS can be optimized from $O(n^2)$ to $O(n \log n)$ via a DP + Binary Search hybrid.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic linear recurrence mechanics	Climbing Stairs (70), Fibonacci Number (509), Min Cost Climbing Stairs (746)
Intermediate	Decision-based and running-extremum DP	House Robber (198), Maximum Subarray (53), Coin Change (322)
Advanced	Multi-state and hybrid DP	Best Time to Buy and Sell Stock with Cooldown (309), Longest Increasing Subsequence (300), Word Break (139)
Expert	Advanced recurrence design, complex multi-state	Longest Valid Parentheses (32), Domino and Tromino Tiling (790), Maximum Alternating Subsequence Sum (1911)

SECTION 24 — Memory Tricks

- **Mnemonic:** “**S**tate, **R**ecurrence, **B**ase case, **O**ptimize” (SRBO) — the four-step DP derivation order.
 - **Visualization:** **Filling in a table of answers left to right**, where each new cell only glances back at a few recent cells, never re-deriving from scratch.
 - **Recognition shortcut:** Recursive brute force with repeated identical calls, describable by one index $\rightarrow$ 1D DP; define state first, always.
-

SECTION 25 — Final Summary

1D Dynamic Programming converts exponential brute-force recursion into polynomial (often linear) time by caching solutions to overlapping subproblems describable by a single index, building each answer from a proven recurrence over strictly smaller, already-solved subproblems. The single most important thing to remember forever: **always define $dp[i]$ ’s precise meaning in plain English before writing the recurrence — this single discipline prevents the vast majority of DP bugs — and always check afterward whether the solution can be space-optimized to $O(1)$ using rolling variables, since most 1D DP recurrences only ever need a small, constant window of prior history.**